

```
import numpy as np
import pandas as pd
```

```
np.random.seed(42)
n=300
```

```
gender = np.random.choice(['Male','Female'],n)
age = np.random.randint(15,20,n)
department = np.random.choice(['Science','Commerce','Arts','Engineering'],n)
attendance = np.round(np.random.uniform(40,100,n),2)
study_hours = np.round(np.random.uniform(0.5, 10, n), 2)
prev_gpa = np.round(np.random.uniform(4.0, 10.0, n), 2)
extra = np.random.choice(['Yes', 'No'], n)
internet = np.random.choice(['Yes', 'No'], n)
sleep_hours = np.round(np.random.uniform(4, 10, n), 2)
part_time = np.random.choice(['Yes', 'No'], n)
```

```
final_score = (
    0.30 * attendance +
    0.25 * (study_hours * 5) +
    0.20 * (prev_gpa * 5) +
    0.10 * (sleep_hours * 3) +
    3 * (extra == 'Yes') +
    (-3) * (part_time == 'Yes') +
    np.random.normal(0, 3, n)
)
final_score = np.round(np.clip(final_score, 55, 100), 2)
```

```
df = pd.DataFrame({
    'Gender': gender,
    'Age': age,
    'Department': department,
    'Attendance_%': attendance,
    'Study_Hours_Per_Day': study_hours,
    'Previous_GPA': prev_gpa,
    'Extracurricular': extra,
    'Internet_Access': internet,
    'Sleep_Hours': sleep_hours,
    'Part_Time_Job': part_time,
    'Final_Score': final_score
})
```

```
for col in ['Attendance_%', 'Study_Hours_Per_Day', 'Sleep_Hours']:
    idx = np.random.choice(df.index, 15, replace=False)
    df.loc[idx, col] = np.nan
```

```
df.to_csv('student_dataset.csv', index=False)
```

```
!pip install reportlab -q
```

2.0/2.0 MB 25.6 MB/s eta 0:00:00

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import r2_score, mean_absolute_error
```

```
np.random.seed(42)
```

```
n = 300
```

```
gender = np.random.choice(['Male', 'Female'], n)
age = np.random.randint(15, 20, n)
```

```

department = np.random.choice(['Science', 'Commerce', 'Arts', 'Engineering'], n)
attendance = np.random.uniform(40, 100, n)
study_hours = np.random.uniform(0.5, 10, n)
prev_gpa = np.random.uniform(4.0, 10.0, n)
extra = np.random.choice(['Yes', 'No'], n)
internet = np.random.choice(['Yes', 'No'], n)
sleep_hours = np.random.uniform(4, 10, n)
part_time = np.random.choice(['Yes', 'No'], n)

```

```

final_score = (
    0.30 * attendance +
    0.25 * (study_hours * 5) +
    0.20 * (prev_gpa * 5) +
    0.10 * (sleep_hours * 3) +
    3 * (extra == 'Yes') +
    (-3) * (part_time == 'Yes') +
    np.random.normal(0, 3, n)
)
final_score = np.clip(final_score, 55, 100)

```

```

df = pd.DataFrame({
    'Gender': gender,
    'Age': age,
    'Department': department,
    'Attendance_%': attendance,
    'Study_Hours_Per_Day': study_hours,
    'Previous_GPA': prev_gpa,
    'Extracurricular': extra,
    'Internet_Access': internet,
    'Sleep_Hours': sleep_hours,
    'Part_Time_Job': part_time,
    'Final_Score': final_score
})

```

```

print("Dataset Shape:", df.shape)
print("\nFirst 5 rows:")
df.head()

```

Dataset Shape: (300, 11)

First 5 rows:

	Gender	Age	Department	Attendance_%	Study_Hours_Per_Day	Previous_GPA	Extracurricular	Internet_Access	Sleep_Hours	Final_Score
0	Male	15	Arts	56.483908	2.520467	7.427914	Yes	Yes	5.061323	68.234567
1	Female	15	Engineering	52.433659	5.394150	7.972672	Yes	Yes	6.322473	72.345678
2	Male	17	Engineering	92.693240	9.767642	7.262673	Yes	No	9.782934	85.456789
3	Male	16	Commerce	85.419948	4.860395	9.186957	No	No	5.491482	78.567890
4	Male	19	Science	42.813788	5.794401	8.403460	Yes	No	4.802822	65.678901

```

for col in ['Attendance_%', 'Study_Hours_Per_Day', 'Sleep_Hours']:
    idx = np.random.choice(df.index, 15, replace=False)
    df.loc[idx, col] = np.nan

```

```

print("Missing values before:\n", df.isnull().sum())

```

```

Missing values before:
Gender          0
Age             0
Department      0
Attendance_%    15
Study_Hours_Per_Day  15
Previous_GPA    0
Extracurricular 0
Internet_Access 0
Sleep_Hours     15
Part_Time_Job   0
Final_Score     0
dtype: int64

```

```

for col in ['Attendance_%', 'Study_Hours_Per_Day', 'Sleep_Hours']:
    df[col].fillna(df[col].median(), inplace=True)

```

```
print("\nMissing values after:\n", df.isnull().sum())
```

Missing values after:

```
Gender      0
Age         0
Department  0
Attendance_% 0
Study_Hours_Per_Day 0
Previous_GPA 0
Extracurricular 0
Internet_Access 0
Sleep_Hours 0
Part_Time_Job 0
Final_Score 0
```

dtype: int64

/tmp/ipykernel_1243/2889246075.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]

```
df[col].fillna(df[col].median(), inplace=True)
```

```
df_model = df.copy()
```

```
for col in ['Gender', 'Department', 'Extracurricular', 'Internet_Access', 'Part_Time_Job']:
```

```
    df_model[col] = df_model[col].astype('category').cat.codes
```

```
scaler = MinMaxScaler()
```

```
scale_cols = ['Attendance_%', 'Study_Hours_Per_Day', 'Previous_GPA', 'Sleep_Hours']
```

```
df_model[scale_cols] = scaler.fit_transform(df_model[scale_cols])
```

```
print("\nPreprocessing complete!")
```

Preprocessing complete!

```
print("\nDescriptive Statistics:")
```

```
df[['Attendance_%', 'Study_Hours_Per_Day', 'Previous_GPA', 'Sleep_Hours', 'Final_Score']].describe().round(2)
```

Descriptive Statistics:

	Attendance_%	Study_Hours_Per_Day	Previous_GPA	Sleep_Hours	Final_Score
count	300.00	300.00	300.00	300.00	300.00
mean	70.71	5.44	6.96	6.92	55.01
std	16.26	2.64	1.71	1.76	0.16
min	40.16	0.50	4.01	4.01	55.00
25%	58.69	3.56	5.42	5.35	55.00
50%	72.10	5.46	7.05	6.93	55.00
75%	84.18	7.58	8.39	8.46	55.00
max	99.86	9.99	9.95	9.95	56.94

```
plt.figure(figsize=(8, 5))
```

```
plt.hist(df['Final_Score'], bins=20, color='steelblue', edgecolor='white')
```

```
plt.axvline(df['Final_Score'].mean(), color='red', linestyle='--', label=f"Mean = {df['Final_Score'].mean():.1f}")
```

```
plt.title('Distribution of Final Scores')
```

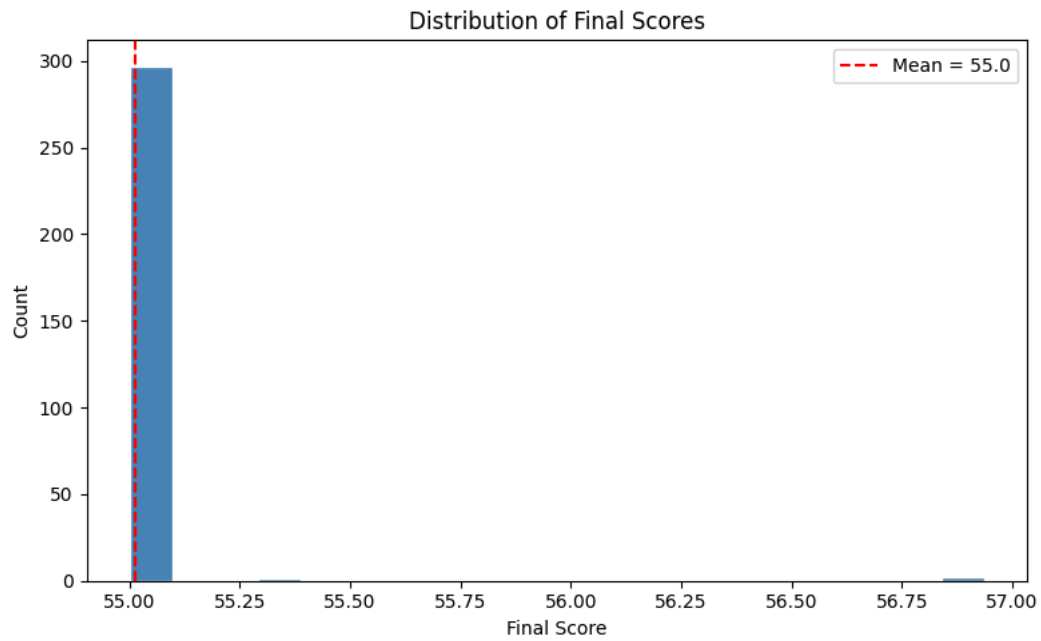
```
plt.xlabel('Final Score')
```

```
plt.ylabel('Count')
```

```
plt.legend()
```

```
plt.tight_layout()
```

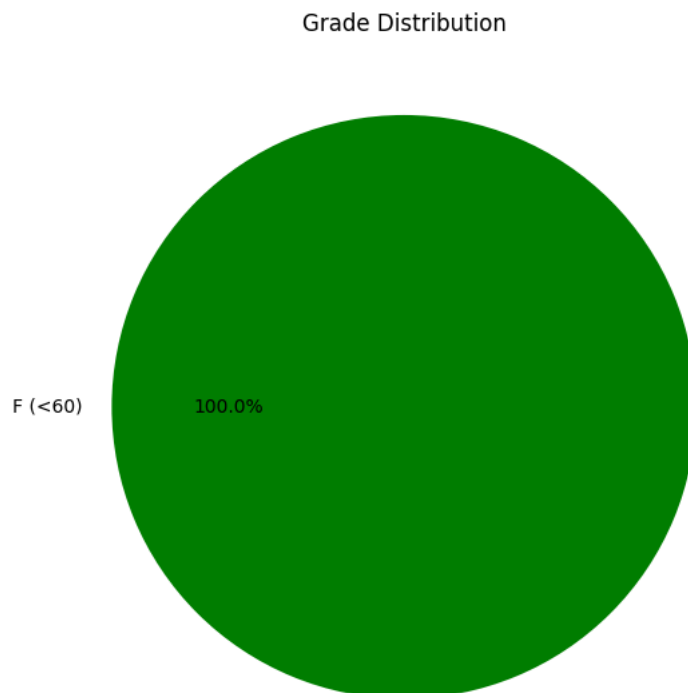
```
plt.show()
```



```
def grade_band(s):  
    if s >= 90: return 'A (90+)'  
    elif s >= 80: return 'B (80-90)'  
    elif s >= 70: return 'C (70-80)'  
    elif s >= 60: return 'D (60-70)'  
    else: return 'F (<60)'
```

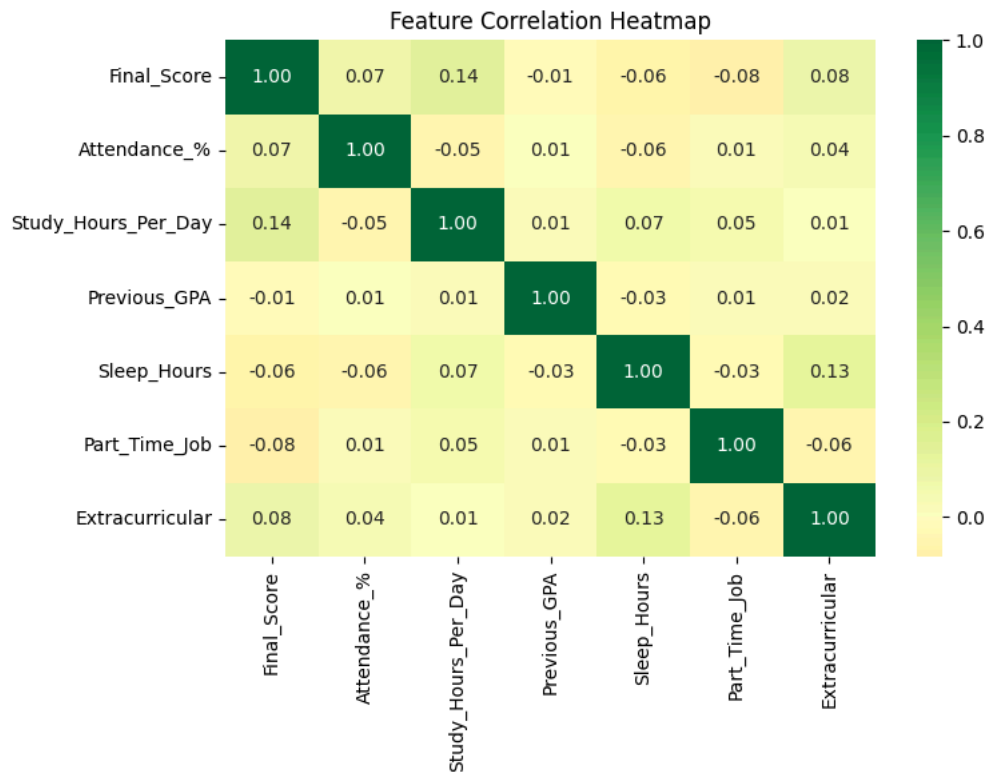
```
df['Grade'] = df['Final_Score'].apply(grade_band)  
grade_counts = df['Grade'].value_counts()
```

```
plt.figure(figsize=(6, 6))  
plt.pie(grade_counts, labels=grade_counts.index, autopct='%1.1f%%',  
        colors=['green', 'steelblue', 'orange', 'salmon', 'red'])  
plt.title('Grade Distribution')  
plt.tight_layout()  
plt.show()
```

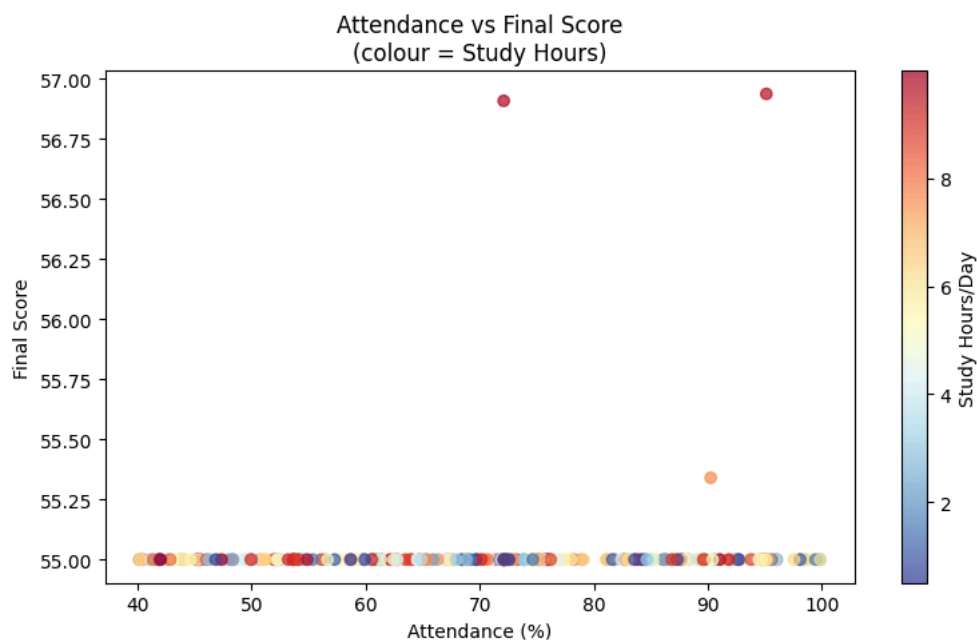


```
corr_cols = ['Final_Score', 'Attendance_%', 'Study_Hours_Per_Day', 'Previous_GPA',
             'Sleep_Hours', 'Part_Time_Job', 'Extracurricular']
corr_df = df_model[corr_cols].corr()
```

```
plt.figure(figsize=(8, 6))
sns.heatmap(corr_df, annot=True, fmt='.2f', cmap='RdYlGn', center=0)
plt.title('Feature Correlation Heatmap')
plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(8, 5))
sc = plt.scatter(df['Attendance_%'], df['Final_Score'],
                 c=df['Study_Hours_Per_Day'], cmap='RdYlBu_r', alpha=0.7)
plt.colorbar(sc, label='Study Hours/Day')
plt.title('Attendance vs Final Score\n(colour = Study Hours)')
plt.xlabel('Attendance (%)')
plt.ylabel('Final Score')
plt.tight_layout()
plt.show()
```

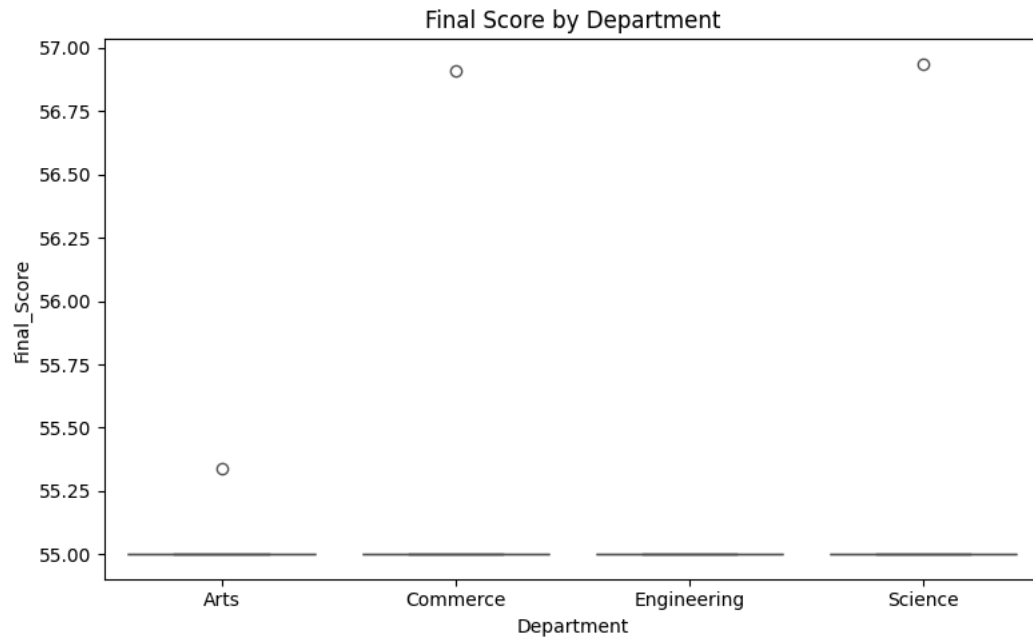


```
plt.figure(figsize=(8, 5))
dept_order = df.groupby('Department')['Final_Score'].median().sort_values(ascending=False).index
sns.boxplot(data=df, x='Department', y='Final_Score', order=dept_order, palette='Set2')
plt.title('Final Score by Department')
plt.tight_layout()
plt.show()
```

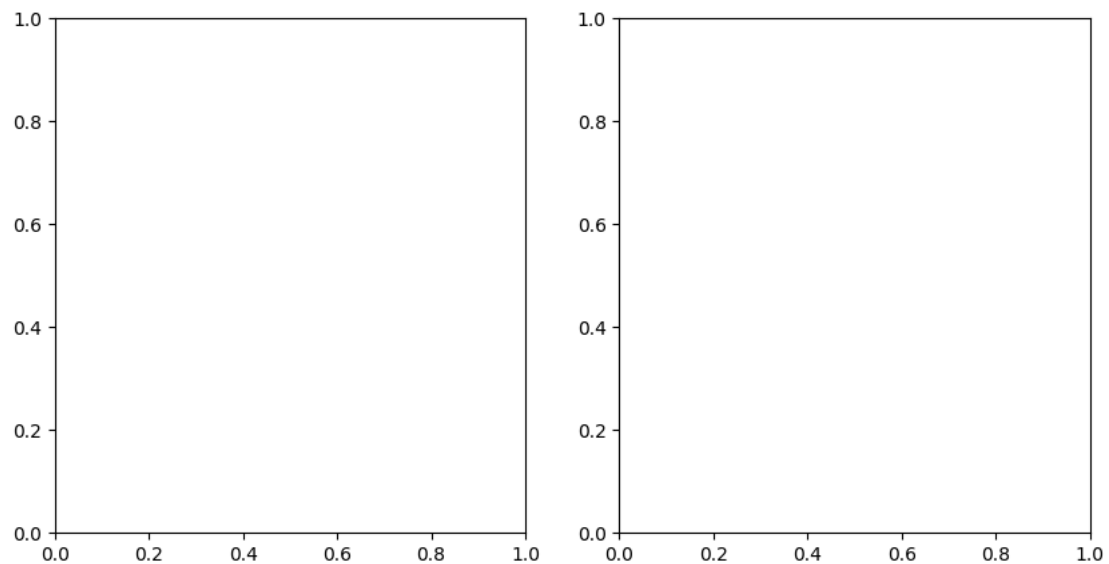
/tmp/ipykernel_1243/3791937429.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

```
sns.boxplot(data=df, x='Department', y='Final_Score', order=dept_order, palette='Set2')
```



```
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
```



```
extra_avg = df.groupby('Extracurricular')['Final_Score'].mean()
axes[0].bar(extra_avg.index, extra_avg.values, color=['green', 'salmon'])
axes[0].set_title('Avg Score vs Extracurricular Activity')
axes[0].set_ylabel('Avg Final Score')
for i, v in enumerate(extra_avg.values):
    axes[0].text(i, v + 0.3, f'{v:.1f}', ha='center')
```

```
pt_avg = df.groupby('Part_Time_Job')['Final_Score'].mean()
axes[1].bar(pt_avg.index, pt_avg.values, color=['green', 'salmon'])
axes[1].set_title('Avg Score vs Part-Time Job')
axes[1].set_ylabel('Avg Final Score')
for i, v in enumerate(pt_avg.values):
```

```
axes[1].text(i, v + 0.3, f'{v:.1f}', ha='center')
```

```
plt.tight_layout()
plt.show()
```

<Figure size 640x480 with 0 Axes>

```
features = ['Attendance_%', 'Study_Hours_Per_Day', 'Previous_GPA', 'Sleep_Hours',
            'Gender', 'Extracurricular', 'Internet_Access', 'Part_Time_Job']
```

```
X = df_model[features]
y = df_model['Final_Score']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

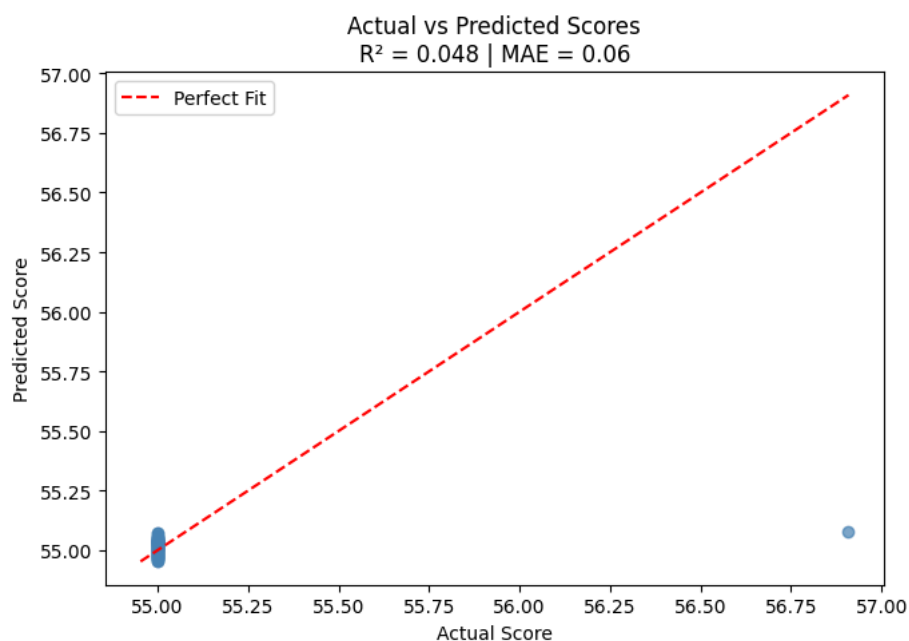
```
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
```

```
print(f"\nModel Performance:")
print(f"  R2 Score : {r2:.4f}")
print(f"  MAE      : {mae:.4f}")
```

```
Model Performance:
R2 Score : 0.0478
MAE      : 0.0585
```

```
plt.figure(figsize=(7, 5))
plt.scatter(y_test, y_pred, alpha=0.7, color='steelblue')
lims = [min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max())]
plt.plot(lims, lims, 'r--', label='Perfect Fit')
plt.title(f'Actual vs Predicted Scores\nR2 = {r2:.3f} | MAE = {mae:.2f}')
plt.xlabel('Actual Score')
plt.ylabel('Predicted Score')
plt.legend()
plt.tight_layout()
plt.show()
```

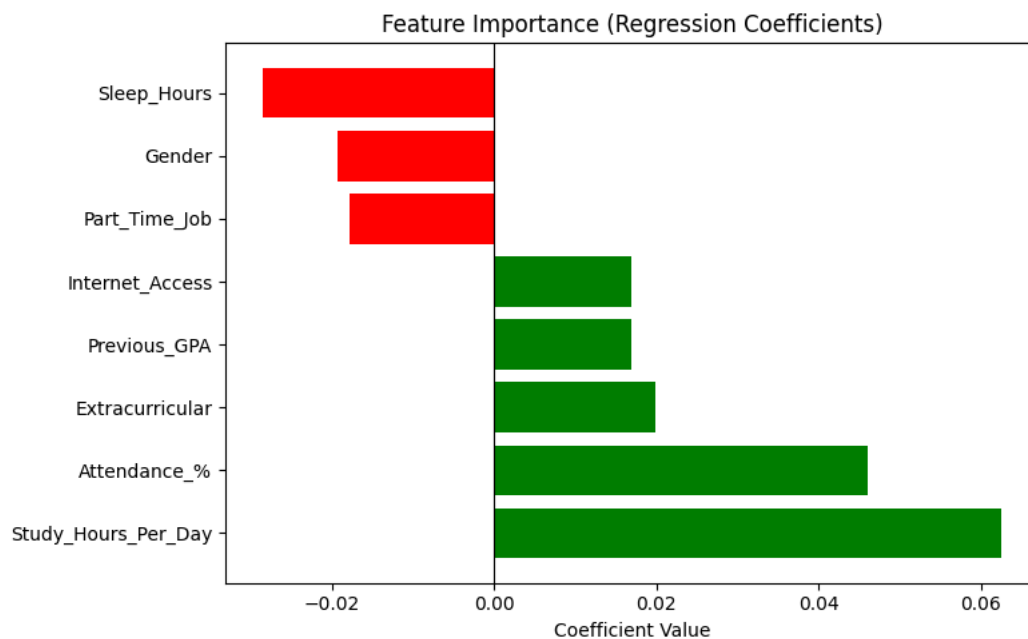


```
coef_df = pd.DataFrame({
    'Feature': features,
    'Coefficient': model.coef_
}).sort_values('Coefficient', ascending=False)
```

```
print("\nFeature Importance (Regression Coefficients):")
print(coef_df.to_string(index=False))
```

```
Feature Importance (Regression Coefficients):
  Feature  Coefficient
Study_Hours_Per_Day    0.062584
Attendance_%           0.046046
Extracurricular        0.019976
Previous_GPA           0.016961
Internet_Access        0.016906
Part_Time_Job          -0.017798
Gender                -0.019308
Sleep_Hours           -0.028529
```

```
plt.figure(figsize=(8, 5))
colors = ['green' if c > 0 else 'red' for c in coef_df['Coefficient']]
plt.barh(coef_df['Feature'], coef_df['Coefficient'], color=colors)
plt.axvline(0, color='black', linewidth=0.8)
plt.title('Feature Importance (Regression Coefficients)')
plt.xlabel('Coefficient Value')
plt.tight_layout()
plt.show()
```



```
print("\n" + "="*55)
print("KEY FINDINGS SUMMARY")
print("="*55)
print(f"  Total Students      : {len(df)}")
print(f"  Avg Final Score     : {df['Final_Score'].mean():.2f}")
print(f"  Model R²            : {r2:.4f}")
print(f"  Model MAE           : {mae:.4f}")
print(f"\n  Top Positive Predictors: Attendance, Study Hours, GPA")
print(f"  Top Negative Predictor : Part-Time Job")
print("="*55)
```

```
=====
KEY FINDINGS SUMMARY
=====
  Total Students      : 300
  Avg Final Score     : 55.01
  Model R²            : 0.0478
  Model MAE           : 0.0585

  Top Positive Predictors: Attendance, Study Hours, GPA
  Top Negative Predictor : Part-Time Job
=====
```


Start coding or [generate](#) with AI.